



Team Project 1- CAP

- Git & GitHub



Version Control

What is “version control”?

Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.

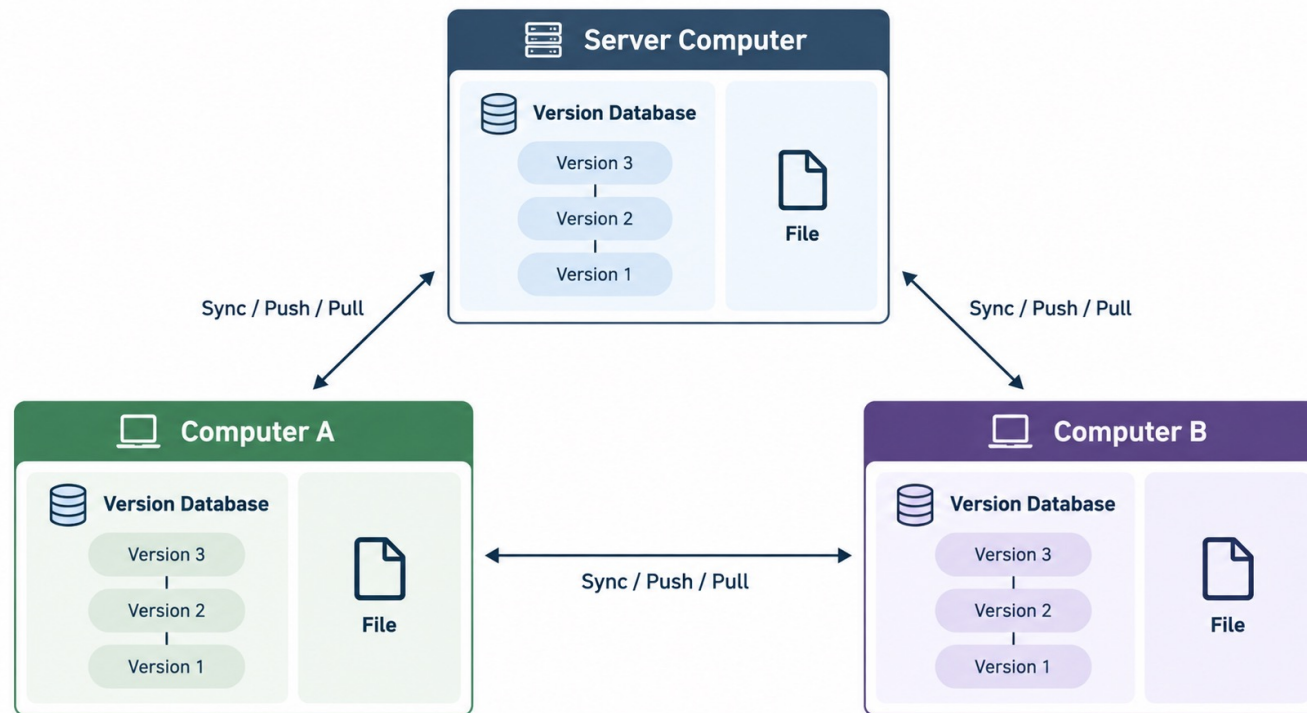
why should you care?

- Revert selected files back to a previous state.
- Revert the entire project back to a previous state.
- Compare changes over time.
- See who last modified something.
- Who introduced an issue and when.



Distributed Version Control Systems

DVCS (such as Git, Mercurial or Darcs), clients don't just check out the latest snapshot of the files; rather, they fully mirror the repository, including its full history.





Introduction to Git

Git is a popular version control system (VCS), it is a free, open source tool that helps you save and manage different versions of your files and code.

- Distributed Version Control System (DVCS).
- Records changes in files over time.
- Allows developers to collaborate.
- Supports branching, merging, and tracking history.
- Tracks changes using **sanpshots** of a project.

Example:

- Team A works on **feature-login**.
- Team B works on **feature-payment**.
- Git allows both teams to work independently and later merge changes.





Introduction to Git

Three Main States:

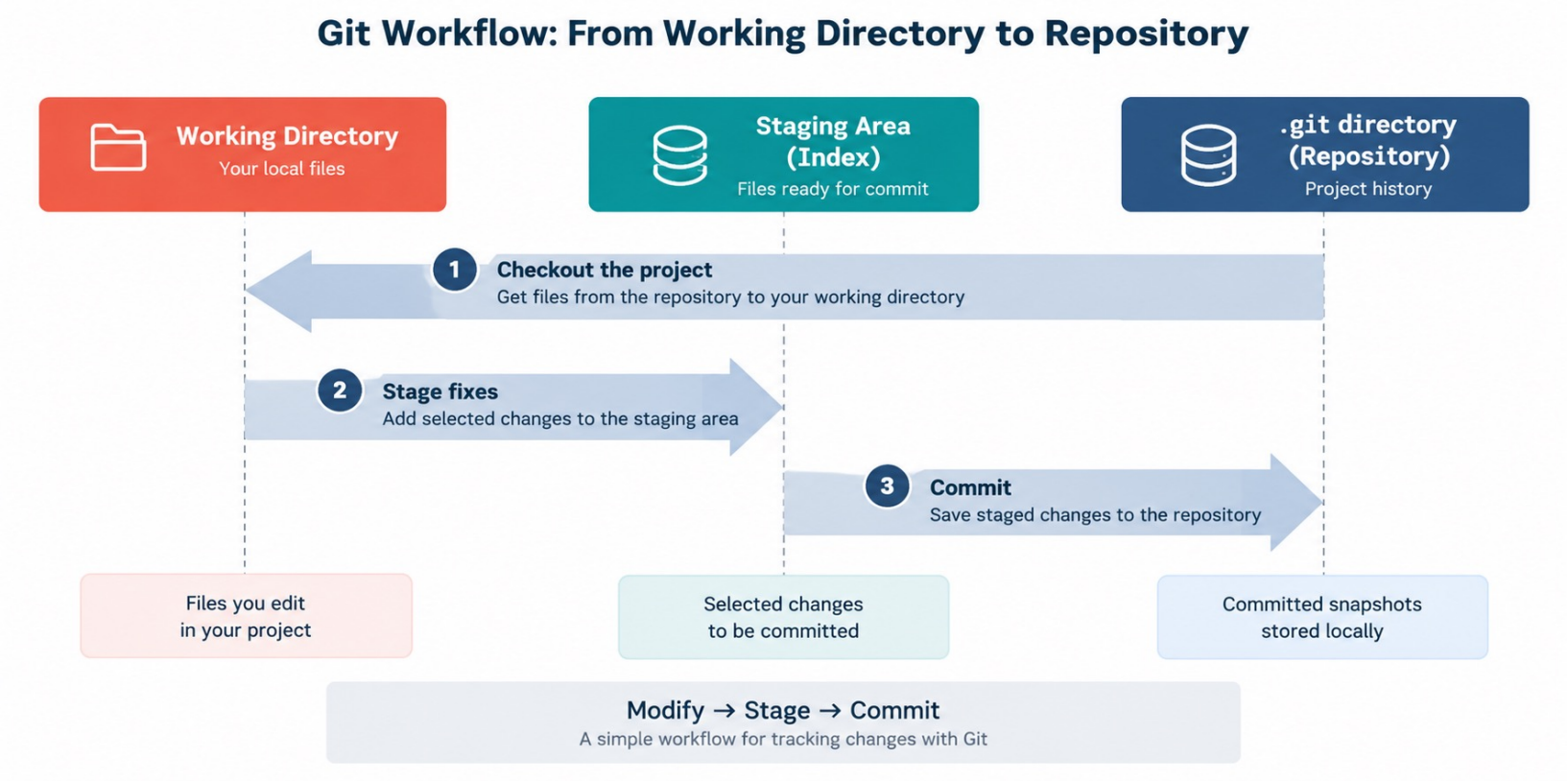
1. Modified → file changed
2. Staged → selected for the next commit
3. Committed → safely stored in the local repository

The basic Git workflow:

1. You modify files in your working tree.
2. You selectively stage just those changes you want to be part of your next commit, which adds only those changes to the staging area.
3. You do a commit, which takes the files as they are in the staging area and stores that snapshot permanently to your Git directory.



Introduction to Git





Install Git

- You can download Git for free from git-scm.com.
- Install package based on OS (Windows, macOS, Linux)

After installation, you will be able to use Git from your terminal or command prompt.

Verify installation:

```
</> Bash
```

```
$ git --version
```

Expected output:

```
</> Bash
```

```
git version 2.45.1
```



Configure Git

The first thing you should do when you install Git is to set your user name and email address. This is important because every Git commit uses this information, and it's immutably baked into the commits you start creating.

Set your user name:

</> Bash

```
$ git config --global user.name "Amjad"
```

Set your email:

</> Bash

```
$ git config --global user.email "amjad@example.com"
```

Verify the configuration:

</> Bash

```
$ git config --list
```




Getting help with Git

If you ever need help while using Git, there are three equivalent ways to get the comprehensive manual page (manpage) help for any of the Git commands:

</> Bash

```
$ git help <command>  
$ git <command> --help  
$ man git-<command>
```

Example:

</> Bash

```
$ git help config
```



Getting Started with Git

You typically obtain a Git repository in one of two ways:

1. You can take a local directory that is currently not under version control, and turn it into a Git repository, or
2. You can clone an existing Git repository from elsewhere.

Initializing a Repository in an Existing Directory

`</> Bash`

```
$ cd path/to/the/directory  
$ git init
```

Cloning an Existing Repository

`</> Bash`

```
$ cd path/to/the/directory  
$ git clone https://github.com/libgit2/libgit2
```



New file in Git

Each file in your working directory can be in one of two states: **tracked** or **untracked**.

- When you create a new file, Git **doesn't** track it automatically.
- Git sees it as Untracked.
- Check with: `git status`

Note: Tracked files are files that were in the last snapshot, as well as any newly staged files; they can be unmodified, modified, or staged. In short, tracked files are files that Git knows about.

Cloning an Existing Repository

`</> Bash`

```
$ echo "Hello Git" > hello.txt  
$ git status
```



Staging Modified Files in Git

Tracking New Files:

In order to begin tracking a new file, you use the command `git add`.

</> Bash

```
$ git add README
```

</> Bash

```
$ git status
```

```
On branch master Your branch is up-to-date with 'origin/master'.
```

```
Changes to be committed:
```

```
29 (use "git restore --staged <file>..." to unstage)
```

```
new file:
```

```
README
```



Staging Modified Files in Git

Staging Modified Files:

If you change a previously tracked file called CONTRIBUTING.md and then run your git status command again, you get something that looks like this:

</> Bash

```
$ git add CONTRIBUTING.md
```

</> Bash

```
$ git status
```

On branch master Your branch is up-to-date with 'origin/master'.

Changes to be committed:

(use "git reset HEAD <file>..." to unstage)

new file:

README

Changes not staged for commit:

(use "git add <file>..." to update what will be committed) (use "git

checkout -- <file>..." to discard changes in working directory)

modified:

CONTRIBUTING.md





Commits in Git

Committing Your Changes:

- A **commit** records a **snapshot of the files currently in the staging area**.
- Files that have not been staged with `git add` are **not included** in the commit.

Commit Staged Changes:

```
</> Bash  
$ git commit
```

Commit with a Message:

```
</> Bash  
$ git commit -m "Fix Login broken link"
```

Example Output:

```
</> Bash  
[master 463dc4f] Fix login broken link  
2 files changed, 2 insertions(+)  
create mode 100644 README
```

Working Directory → `git add` → **Staging Area** → `git commit` → **Repository**

Only staged changes are committed. Each commit creates a snapshot that becomes part of the project's history.





Deleting files with Git

Git can track **file deletions** just like other changes. A deleted file is not removed from previous commits - it remains available in the repository history.

Delete & Stage a File: *(The recommended Git command)*

</> Bash

```
$ git rm notes.txt
```

This performs **two actions**: **Deletes the file + Stages the deletion**
Then commit the change

Deleting file manually:

- Git detects the deletion
- But you still need to stage it
- Then commit



Restoring deleted files with Git

Before staging:

</> Bash

```
$ git restore notes.txt
```

If the deletion is already staged:

</> Bash

```
$ git restore --stages notes.txt
```

```
$ git restore notes.txt
```

Deleting a tracked file does not erase its history. Git records the deletion as a new change, while previous versions remain in earlier commits.



Git Tag

- The `git tag` command is used in Git to create, list, delete, or verify tags.
- Tags are essentially labels or bookmarks for specific points in your project's history, often used to mark releases.
- Label a commit (for versions/releases)

Creating Tag:

</> Bash

```
$ git tag v1.0
//or using annotated tag
$ git tag -a v1.0 -m "my version 1.4"
```

listing Tags:

</> Bash

```
$ git tag
```

V1.0

Deleting Tag:

</> Bash

```
$ git tag -d v1.0
```

Show Tag:

</> Bash

```
$ git show v1.0
tag v1.0
Tagger: Ben Straub <ben@straub.cc>
Date: Sat May 3 20:19:12 2014 -0700
```



Git History (git log)

- Git keeps a history of every commit.
- The `git log` command lets you explore **what changed, who changed it, and when**.
- The `git log --oneline` display one commit per line.

View commit history:

```
</> Bash
```

```
$ git log
```

```
</> Bash
```

```
commit a3f5c81...  
Author: Amjad <amjad@example.com>  
Date: Tue Sep 15 10:30:00 2026
```

```
Add login page
```

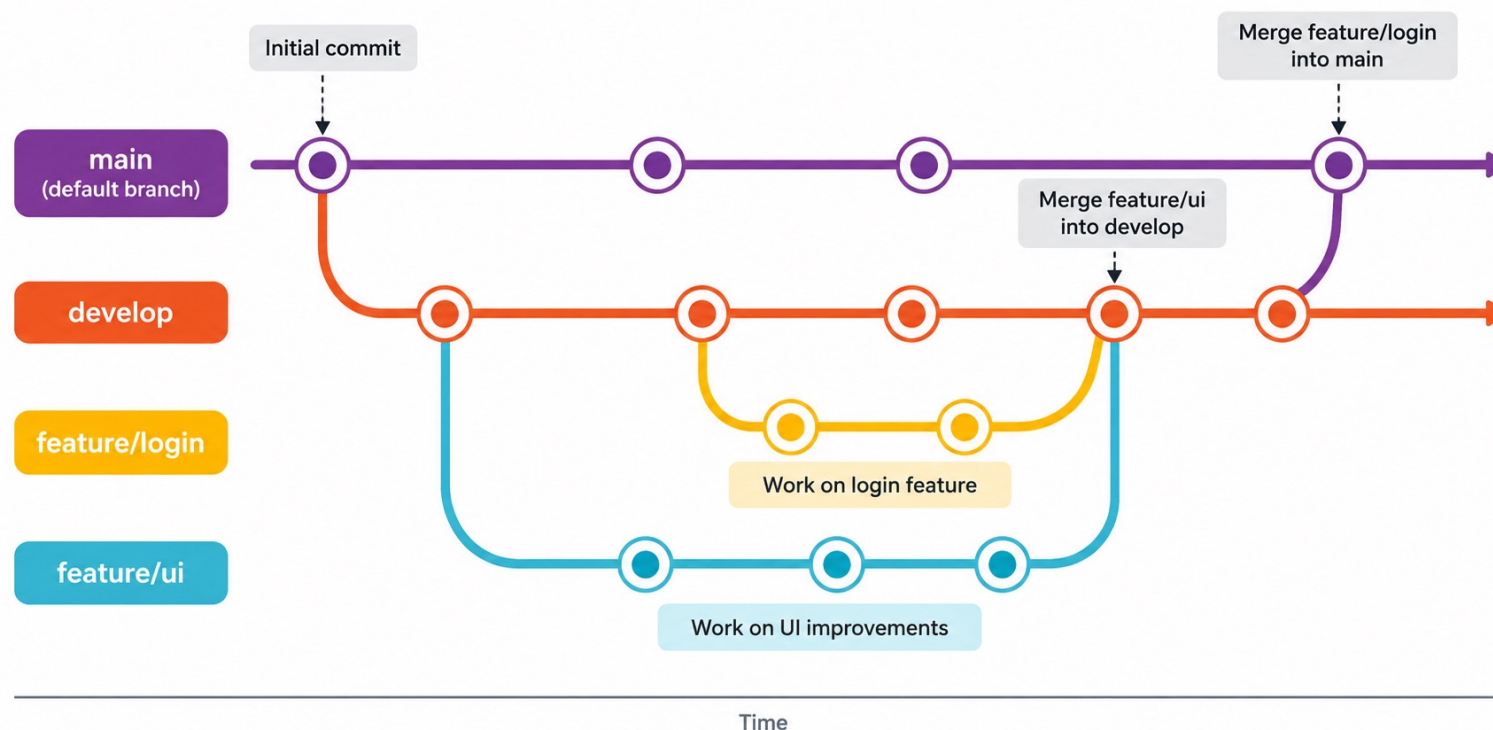
Each commit contains:

- Commit ID (Hash) → uniquely identifies the commit
- Author → who created it
- Date → when it was committed
- Message → describes the change



Git Branching

- Independent line of development.
- Default branch: **main** / **master**.
- Create: **git branch feature-x**
- Switch branch: **git checkout feature-x** (for modern versions: **git switch feature-x**).





Git Branching

Create New Branch / Switch to the branch :

</> Bash

```
$ git branch feature-login
$ git switch feature-login
# or simply you can create and switch in one command
$ git switch -c feature-login
```

List Branches :

</> Bash

```
$ git branch
# List all branches
  feature-login
* Master
  Development
```

Create / Switch (old version) :

</> Bash

```
$ git branch feature-login
$ git checkout feature-login
# or simply you can create and switch in one command
$ git checkout -b feature-login
```



Git Merge

Merging combines changes from one branch into another while preserving the development history.

1. Switch to the Destination Branch

First, switch to the branch that should **receive the changes**:

```
</> Bash
$ git switch main
```

2. Merge the Feature Branch

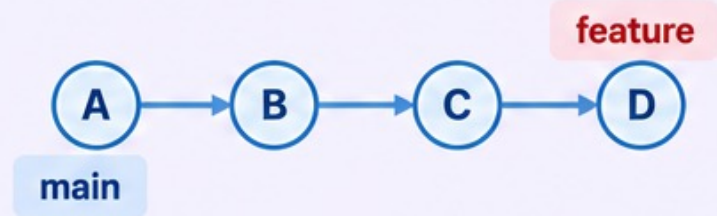
```
</> Bash
$ git merge feature-branch
```

Fast-Forward Merge

If main has **not changed** since the feature branch was created, Git can simply move main forward:

Fast-forward merge

If the target branch has not moved, Git may perform a fast-forward merge without creating a merge commit.



In this case, main simply moves forward to the latest commit.

1 Before merge

You have a feature branch with new commits, based on main.

```
graph LR
    A((A)) --> B((B))
    B --> C((C))
    C --> D((D))
    subgraph main
    A
    B
    end
    subgraph feature
    C
    D
    end
```

A - Initial commit
B - Common commit (branch point)
C, D - New commits in feature branch

2 Run the merge

Switch to the branch that will receive the changes (e.g., main) and merge the feature branch.

```
git switch main
git merge feature
```

This creates a new merge commit that combines the changes from both branches.

3 After merge

Both branches are now merged. The history of both branches is preserved.

```
graph LR
    A((A)) --> B((B))
    B --> C((C))
    C --> D((D))
    B --> M((M))
    D --> M
    subgraph main
    A
    B
    M
    end
    subgraph feature
    C
    D
    end
```

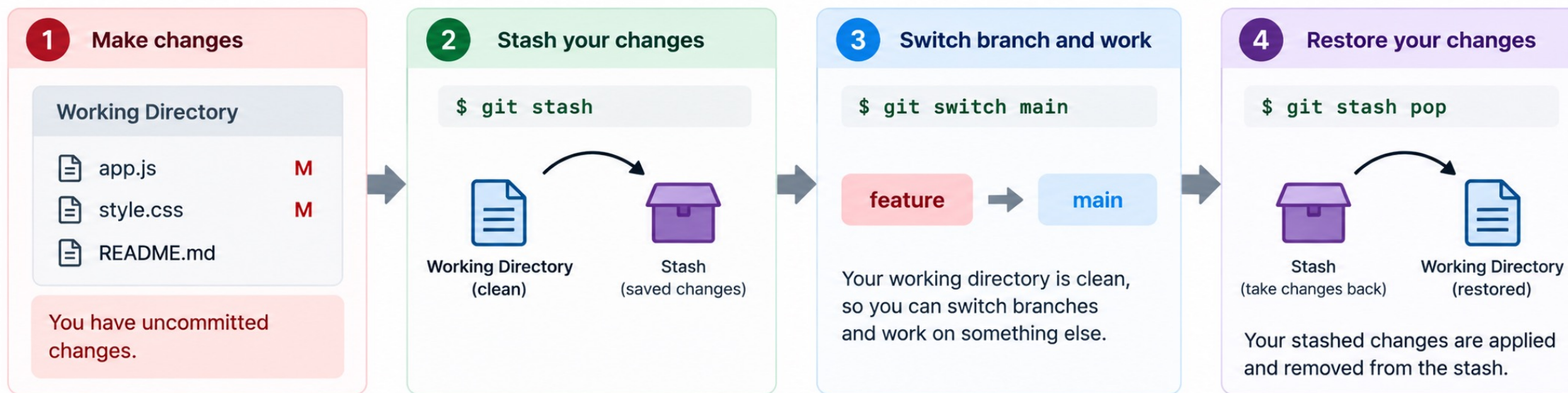
M - Merge commit (has two parents: B and D)



Git Stash

- **git stash** temporarily save uncommitted changes so you can work on something else without creating a commit.
- Useful when switching branches.

Git Stash – Workflow





Git Stash

Stash Commands:

</> Bash

```
$ git stash # your changes Stash
$ git stash push -m "My Message" # Stash with a message
$ git stash list # List all stash
$ git stash branch <branchName> # Create a branch from a stash
$ git stash apply # Restore your most recent stashed changes (keeps the stash in the stack)
$ git stash apply stash@{n} # Restore a specific stash from the list
$ git stash pop # apply the latest stash and remove it from the stack
$ git stash drop stash@{n} # Delete a specific stash when you no longer need it.
```




Git Undo

Git provides different ways to undo changes depending on **where the change currently exist.**

1. `git restore`
2. `git revert`
3. `git reset`
4. `git commit --amend`
5. `git rebase`
6. `git reflog`

Key idea: First identify whether the change is **modified, staged, or committed**, then choose the appropriate undo operation.



Git Restore

Use `git restore` when you modified a tracked file in the **Working Directory**, but **have not staged the changes**, and you want to discard those modifications.

```
</> Bash
```

```
$ git status  
Changes not staged for commit:  
  modified: README.md
```



```
</> Bash
```

```
$ git restore README.md
```



```
</> Bash
```

```
$ git status  
nothing to commit, working tree clean
```

The uncommitted modifications are **discarded**.

Stage: Working Directory

Purpose: Undo modifications that have **not been staged**.



Git Restore Stages

You already staged a file, but you don't want the file to be included in the next commit.

`</> Bash`

```
$ git add README.md  
$ git status
```

Changes to be committed:
modified: README.md



`</> Bash`

```
$ git restore --staged README.md
```

The changes are **not deleted**.
They return to:



`</> Bash`

```
$ git status
```

Changes not staged for commit:
modified: README.md

`git restore file.txt` → Discard file Changes

`git restore --staged file.txt` → Unstage changes, but keep them

Stage: Staging area

Purpose: Remove something from the next commit **without losing your work**.



Git commit --amend

The commit has already happened, but you notice that the **most recent commit** has a mistake.

Common situations:

- Forgot to add a file
- Forgot some changes
- Wrong commit message

```
</> Bash
```

```
$ git commit -m "Add Login Page"
```



```
</> Bash
```

```
$ git add forgotten-file.txt  
$ git commit --amend --no-edit
```

--no-edit keeps the existing commit message.

--amend does **not edit the existing commit in place**. It replaces it with a new commit.

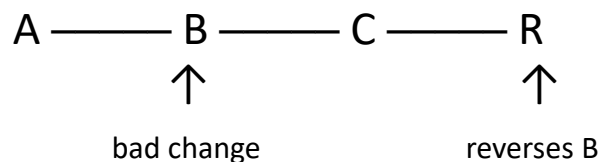
Stage: Latest Commit

Purpose: Correct or complete the **most recent commit**.



Git revert

A commit has already been made - especially when it has been **pushed/shared** - and you want to undo its effect **without deleting or rewriting history**.



```
</> Bash
```

```
$ git log -oneline
```

```
83ab210 Add dashboard
72fc491 Add broken validation
41de802 Initial project
```

```
</> Bash
```

```
$ git revert 72fc491
```

Nothing disappears from the history.

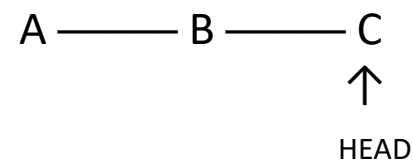
Stage: Committed / Shared History

Purpose: Safely reverse an existing commit **by creating another commit**.

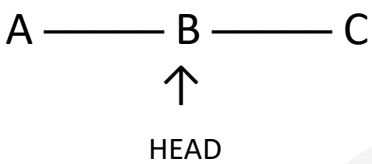


Git reset

- `git reset` moves the **current branch pointer (HEAD)** to another commit.
- It is especially useful for undoing **local commits that have not been shared**.



```
</> Bash
$ git reset <option> HEAD~1
```



Command	Commit	Staging Area	Working Files
--soft	Undo	Keep staged	Keep
--mixed	Undo	Unstage	Keep
--hard	Undo	Clear	Discard

Stage: Local Commit History
Purpose: Move the branch backward and optionally preserve or discard the associated changes.





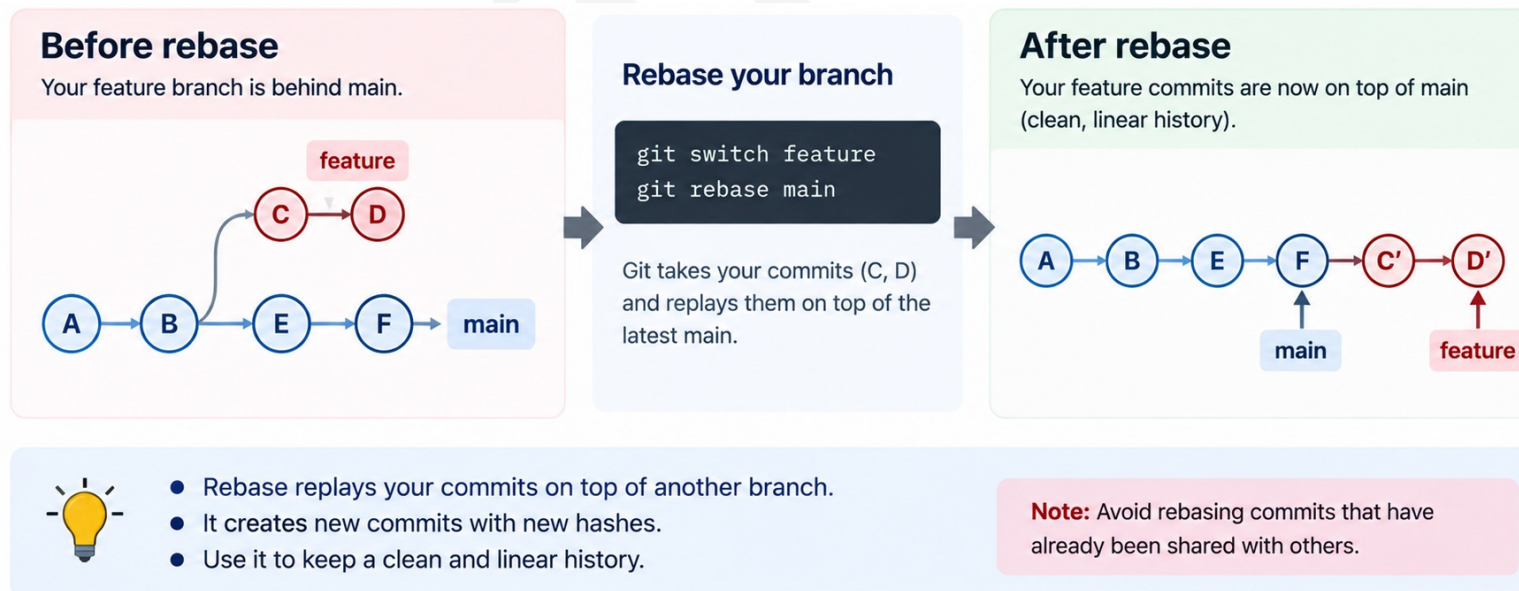
Git rebase

- **git rebase** moves your branch to a new base and **replays its commits on top of another branch**, creating a cleaner, linear history.

Use `rebase` when you want to:

- Update your feature branch with the latest changes from `main`.
- Keep the project history **clean and linear**.
- Reorganize local commits before sharing them.

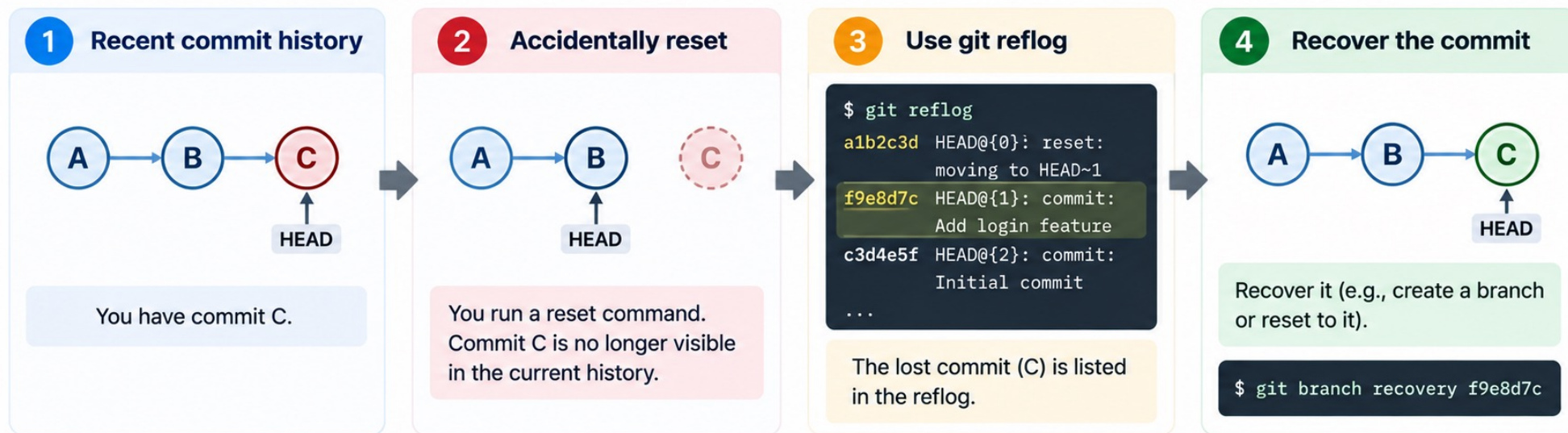
```
</> Bash  
$ git rebase --continue
```





Git reflog

- What if you make a mistake while using:
 - git reset
 - git rebase
 - git commit --amend



git reflog keeps a record of where your local HEAD and references have been, so you can find and recover commits that seem to be lost (e.g., after reset, rebase, or amend).



Git .gitignore

- A **.gitignore** file tells Git which files or folders should **not be tracked**.

Why Use .gitignore?

Common files that should not be committed:

- Build/generated files
- IDE/editor settings
- Log and temporary files
- Dependencies
- Environment/configuration files containing secrets

```
<> gitignore
# Ignore log files
*.log

# Ignore dependencies
node_modules/

# Ignore environment variables
.env

# Ignore build output
dist/

# Ignore IDE settings
.idea/
```

`.gitignore` prevents unwanted untracked files from being added to Git; it does not erase files or remove existing tracked history.





Git .gitignore

Common Patterns

Pattern	Meaning
*.log	Ignore all .log files
temp/	Ignore the entire temp directory
.env	Ignore a specific file
*.tmp	Ignore all temporary files
!important.log	Do not ignore this matching file

Important: Already Tracked Files

`.gitignore` does **not** automatically stop tracking a file that has already been committed.

To stop tracking it while keeping it locally:

```
</> Bash
$ git rm --continue .env
```

Then commit the change:

```
</> Bash
$ git commit -m "Stop tracking .env file"
```





GitHub

- **GitHub** is a cloud-based platform for **hosting Git repositories** and collaborating on software projects.
- It uses **Git** for version control and adds collaboration and project-management features such as:
 - ☁ Remote repository hosting
 - 👥 Team collaboration
 - 🔀 Pull Requests & code reviews
 - 🐛 Issues & project tracking
 - ⚙ Automation with GitHub Actions
 - 🔒 Access control and repository management

You can use **Git without GitHub**, and GitHub is one of several services that can host and collaborate around Git repositories.





Git VS Github

 Git	 GitHub
Version control system	Cloud platform built around Git
Installed on your computer	Primarily accessed through the web and Git
Tracks versions and changes	Hosts and shares Git repositories
Works completely offline	Requires a network for remote operations
Provides commits, branches, merges, tags, etc.	Adds Pull Requests, Issues, Actions, collaboration, etc.
Open-source software	Service/platform operated by GitHub

How They Work Together

You use Git locally, and GitHub hosts your repositories and enables collaboration.





Working with Github

Git uses **remote repositories** such as GitHub to share code and synchronize changes between developers.

git clone - Copy a Repository

Downloads an existing remote repository to your computer, including its Git history.

```
</> Bash  
$ git clone <repository-url>
```

git push - Send Your Commits

Uploads your **local commits** to the remote repository.

```
</> Bash  
$ git push origin main
```

git pull - Get Latest Changes

Downloads changes from the remote repository and integrates them into your current branch.

```
</> Bash  
$ git pull origin main
```



Github GUI Clients

Git GUI clients provide a visual interface for working with Git and GitHub, allowing you to perform common Git operations without typing every command in the terminal.

What Can You Do with a GUI Client?

-  Clone and manage repositories
-  View changed files and differences
-  Stage and commit changes
-  Create and switch branches
-  Merge branches and resolve conflicts
-  Push changes to GitHub
-  Pull / fetch remote changes
-  Browse commit history visually

Common Git GUI Clients

- **GitHub Desktop** — GitHub's official desktop client; simple and beginner-friendly.
- **Visual Studio Code** — Built-in Source Control interface for Git, with GitHub integration and extensions.
- **Sourcetree** — Visual Git client with detailed branch and commit-history visualization.
- **GitKraken** — Git client with strong visual tools for branches, merges, and repository history.

Author

Amjad Ataallah

Email: amjad.atallah@najah.edu
Linkedin: <https://www.linkedin.com/in/ataallah/>
Website: <https://amjadataallah.github.io>



Yanal Sweseh

Email: yanal.swaseh@najah.edu
Linkedin: <https://www.linkedin.com/in/yanalsweseh/>
Website: <https://github.com/yswesieh>

